SACM — Software Artifact Composition Module

SACM — Software Artifact Composition Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Creation Standard (SCS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Artifact Composition Governance

Module ID: SLGA-SCS-SACM-004

Origin ID: OOF.CM.SLGA.02.04.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 11.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Software Artifact Composition Module (SACM) governs how software

artifacts are selected, associated, combined, integrated, transformed, and recognized as constituent elements of a composite software artifact during software creation.

SACM establishes the methodological conditions under which the composition

of multiple software artifacts may be recognized as a governed creation activity, with identifiable constituent artifacts, legitimate composition authority, defined integration boundaries, supported compatibility relationships, and reconstructable composition evidence.

It governs the relationship between the intended composite software object,

the artifacts contributing to its formation, their respective identities and integrity states, the mechanisms used to combine them, the resulting composite artifact, and the evidence supporting the composition process.

SACM distinguishes technical assembly from governed composition, the

integrity of individual artifacts from the integrity of their combined result, and successful integration from evidence that the resulting software composition remains within the applicable creation requirements and authority boundaries.

It applies to manually assembled software, generated applications, modular

systems, compiled software, AI-generated components, autonomous development outputs, reused software, hybrid human–AI artifacts, and multi-agent software creation processes.

Its principal output is:

Governed Software Artifact Composition State

Its foundational distinction is:

Technical Artifact Assembly ≠ Governed Software Artifact Composition.

Operational Role

The operational role of SACM is:

To ensure that software artifacts are combined into identifiable composite software objects through legitimate, bounded, compatible, integrity-aware, and evidence-supported composition processes that preserve the relationships between constituent artifacts and the resulting software creation output.

SACM governs the formation of software artifacts whose identity or function

depends on the combination of multiple constituent artifacts.

It establishes whether the composition process is sufficiently supported by evidence and whether material constituent relationships remain identifiable.

It does not assume that combining individually valid components automatically produces a valid composite artifact.

It also does not assume that successful compilation, packaging, or execution establishes the legitimacy of the composition process.

Position in the SCS Internal Governance Flow

The SCS internal governance flow is:

Development Method Selection → Code & Artifact Integrity → Autonomous

Software Creation Control → Software Artifact Composition → Development Traceability

SACM governs:

Software Artifact Composition

It is the fourth Core Module of the Software Creation Standard (SCS).

Its principal upstream references include:

Governed Software Requirements Baseline

from Software Requirements Standard (SRS).

Governed Development Method Selection State

from DMSM — Development Method Selection Module.

Governed Code & Artifact Integrity State

from CRIM — Code & Artifact Integrity Module.

Governed Autonomous Software Creation Control State

from ASCCM — Autonomous Software Creation Control Module, where autonomous execution is involved.

Its principal output is:

Governed Software Artifact Composition State

That state supports:

DTM — Development Traceability Module

and the establishment of the broader:

Governed Software Creation State

SACM may operate iteratively throughout software creation.

A composite artifact may be assembled, modified, decomposed, regenerated, and recomposed multiple times before the relevant creation activity is completed.

Central Operational Question

How can multiple software artifacts be combined into a coherent software creation output while preserving constituent identity, composition authority, integration boundaries, compatibility evidence, resulting artifact integrity, and reconstructable relationships between the parts and the whole?

Software Artifact Composition Governance Object

Every SACM assessment shall identify the Software Artifact Composition

Governance Object.

The object represents a defined composition activity or composite software

artifact whose formation is subject to governance.

It should identify:

Composition Identity: The identifiable composition activity or assessment.

Composition Objective: The intended combined software outcome.

Composite Artifact Identity: The resulting software artifact or intended

artifact.

Requirements Reference: The applicable governed requirements basis.

Constituent Artifacts: The material artifacts participating in

composition.

Constituent Roles: The roles of those artifacts within the composition.

Composition Method: The method used to combine or integrate them.

Composition Authority: The applicable authorization basis.

Composition Boundaries: The permitted integration scope and conditions.

Compatibility Conditions: Material interface and integration requirements.

Composition Evidence: Evidence supporting the formation of the composite

artifact.

Composition State: The current governance condition of the composition.

The composition governance object shall remain distinguishable from the

individual artifacts participating in it.

Constituent Artifact Identity ≠ Composite Artifact Identity.

Composite Software Artifact Identity

SACM shall establish an identifiable reference for the resulting composite

software artifact.

A composite artifact may represent:

• An application

• A software service

• A library

• An executable package

• A software subsystem

• A generated application bundle

• A compiled software product

• An integrated component

• A multi-module software artifact

Composite identity shall not depend exclusively on a product name,

directory structure, or build label.

The same product name may represent materially different compositions.

Product Name ≠ Composite Artifact Identity.

Constituent Artifact Identification

SACM shall identify the material artifacts participating in a composition.

These may include:

• Human-authored code

• AI-generated code

• Reused internal components

• External software components

• Generated interface definitions

• Build artifacts

• Executable modules

• Scripts

• Software templates

• Intermediate representations

• Generated resources

• Software configuration artifacts where relevant to composition

Constituent identification should preserve the artifact identity and version references required to reconstruct the composition.

A generic component name may be insufficient where multiple versions exist.

Component Name ≠ Constituent Artifact Identity.

Constituent Artifact Role

SACM shall identify the relevant role of each material constituent artifact.

Possible roles include:

Functional Component: Implements a defined software capability.

Interface Component: Establishes interaction between software elements.

Supporting Component: Provides required supporting functionality.

Generated Component: Is produced through an automated generation process.

Transformation Input: Contributes to the creation of another artifact.

Composition Output: Results from an assembly or integration activity.

Shared Component: Is reused across multiple composite artifacts.

Execution Component: Contributes to executable software behavior.

The role of a constituent artifact may change across composition states.

Artifact Participation ≠ Fixed Artifact Role.

Composition Objective Governance

SACM shall preserve the relationship between composition and the intended

software creation objective.

The objective should identify:

• The intended composite software function

• Applicable requirements

• Relevant scope

• Material integration expectations

• Necessary constituent relationships

• Applicable constraints

The mere availability of multiple components does not establish that combining them is appropriate.

Component Availability ≠ Composition Necessity.

Composition shall remain connected to the governed creation objective.

Requirements-to-Composition Alignment

SACM shall assess whether the proposed composition remains aligned with the

applicable:

Governed Software Requirements Baseline

Relevant considerations may include:

• Required functionality

• Interface requirements

• Integration constraints

• Data-flow expectations

• Performance-related requirements

• Reliability requirements

• Material architectural constraints

• Relevant evidence obligations

A composition may function technically while failing to represent the intended requirements.

Working Composition ≠ Requirements-Aligned Composition.

Detailed requirements legitimacy and acceptance remain within Software Requirements Standard (SRS).

Composition Scope

SACM shall establish the boundaries of the composition activity.

Composition scope may identify:

• Included artifacts

• Excluded artifacts

• Permitted transformations

• Authorized integration targets

• Relevant repositories

• Permitted build environments

• Approved component versions

• Material external interfaces

Scope shall be sufficiently defined to distinguish authorized composition

from unrecognized expansion.

Composition Scope ≠ Unlimited Integration Authority.

Composition Authority

SACM shall identify the legitimate authority basis for combining material

software artifacts.

Relevant authority conditions may include:

• Authorized creation objective

• Permitted artifact selection

• Permitted component transformation

• Integration authority

• Relevant delegated authority

• Applicable restrictions

• Required approvals

A developer or autonomous agent may possess technical access to multiple components without authority to combine them into a particular software product.

Artifact Access ≠ Composition Authority.

SACM does not independently create organizational or legal authority.

Constituent Inclusion Authority

SACM shall distinguish authority to perform composition from authority to

include a particular constituent artifact.

For example, an autonomous agent may be authorized to assemble an application while remaining prohibited from incorporating restricted internal software or unapproved external components.

Each material inclusion may require a competent basis.

Composition Authority ≠ Unlimited Constituent Inclusion Authority.

The applicable dependency, ownership, licensing, and security conditions

remain governed by their competent domains.

Composition Method Governance

SACM shall identify the method used to combine constituent artifacts.

Possible methods include:

• Manual integration

• Automated assembly

• Compilation

• Linking

• Packaging

• Code generation

• Software synthesis

• Model-driven composition

• AI-assisted integration

• Autonomous multi-agent composition

• Hybrid integration

The selected composition method shall remain connected to the applicable

development method governance state.

A technically available composition mechanism is not automatically suitable or authorized.

Composition Method Availability ≠ Governed Composition Method Use.

Composition Structure

SACM shall preserve the material structural relationships between

constituent artifacts.

These may include:

• Component membership

• Parent–child artifact relationships

• Module relationships

• Interface connections

• Generated artifact dependencies

• Integration boundaries

• Transformation relationships

• Resulting composite identity

Composition structure may be represented as a hierarchy, graph, manifest, or another suitable form.

SACM does not prescribe one universal representation.

Artifact Collection ≠ Governed Composition Structure.

Constituent Relationship Governance

SACM shall identify the material relationships through which constituent artifacts form a composite artifact.

These may include:

Contains: A composite artifact incorporates a constituent.

Generates: An artifact or mechanism produces another artifact.

Transforms: A source artifact is converted into another artifact.

Links: Multiple artifacts are connected through a defined integration mechanism.

Implements: An artifact contributes to a defined software capability.

Interfaces With: An artifact interacts with another constituent.

Replaces: One artifact supersedes another within a composition.

Derives From: An artifact is formed using another artifact as a material source.

Relationship labels shall reflect supported composition reality rather than

inferred associations alone.

Recorded Relationship ≠ Verified Composition Relationship.

Constituent Artifact Integrity

SACM shall preserve relevant integrity states of material constituent

artifacts.

Its primary integrity reference is:

Governed Code & Artifact Integrity State

from CRIM — Code & Artifact Integrity Module.

Relevant considerations may include:

• Artifact identity

• Version integrity

• Origin evidence

• Content integrity

• Transformation history

• Material integrity gaps

• Relevant uncertainty

A constituent artifact with unresolved integrity conditions may affect the composite artifact's governance state.

Constituent Inclusion ≠ Constituent Integrity Acceptance.

SACM does not replace CRIM's competent integrity assessment.

Composite Artifact Integrity

SACM shall distinguish the integrity of constituent artifacts from the

integrity of their combined result.

Even where every constituent artifact has sufficient integrity evidence, the composition process may introduce:

• Incorrect combinations

• Unrecognized transformations

• Artifact substitutions

• Version mismatches

• Missing components

• Unexpected generated outputs

• Unrecorded integration changes

The resulting composite artifact requires its own relevant identity and

integrity assessment.

Component Integrity ≠ Composite Artifact Integrity.

Detailed artifact integrity determination remains within CRIM.

Composition Compatibility

SACM shall assess whether material constituent artifacts can be combined under the applicable composition conditions.

Compatibility may concern:

• Interfaces

• Data structures

• Component versions

• Execution environments

• Build conditions

• Integration protocols

• Relevant runtime assumptions

• Required artifact formats

Compatibility shall be evaluated against the actual intended composition.

Two artifacts may be individually functional while remaining incompatible when combined.

Individual Component Functionality ≠ Composition Compatibility.

Interface Compatibility

SACM shall preserve material interface relationships between constituent

artifacts.

Relevant interface conditions may include:

• Input and output structures

• Data types

• API contracts

• Communication protocols

• Error handling

• Version expectations

• Relevant behavioral assumptions

An interface may be syntactically compatible while its semantic expectations differ.

Syntactic Interface Compatibility ≠ Semantic Interface Compatibility.

SACM shall preserve material unresolved interface assumptions for competent

assessment.

Version Compatibility

SACM shall identify the versions of constituent artifacts used in a

composition.

Version compatibility may be affected by:

• Interface changes

• Removed functionality

• Modified data structures

• Dependency changes

• Build assumptions

• Generated code differences

• Runtime requirements

The existence of version labels does not independently establish compatibility.

Version Identification ≠ Version Compatibility.

Where a composition changes its constituent versions, material compatibility may require reassessment.

Composition Dependency Relationships

Composite software artifacts may depend on internal or external components.

SACM shall identify material dependency relationships necessary to

reconstruct the composition.

Relevant information may include:

• Dependency artifact identity

• Constituent role

• Version reference

• Inclusion mechanism

• Relevant compatibility assumptions

• Composition evidence

SACM governs how dependency artifacts participate in composition.

The broader governance of dependency legitimacy, continuity, risk, and

lifecycle remains within Software Dependency Standard (SDS).

Dependency Inclusion ≠ Dependency Governance Acceptance.

External Software Inclusion

SACM applies where externally sourced software is incorporated into a composite artifact.

Relevant conditions may include:

• External artifact identity

• Origin references

• Version

• Inclusion scope

• Transformation conditions

• Compatibility

• Applicable authority references

• Material integrity evidence

A publicly available component is not automatically authorized for every intended composition.

Public Availability ≠ Authorized Software Inclusion.

Licensing and ownership determinations remain within competent legal and identity-governance processes.

AI-Generated Constituent Artifacts

SACM applies to AI-generated software components incorporated into

composite artifacts.

Relevant composition conditions may include:

• Generated artifact identity

• Generation origin

• Relevant input references

• Applicable requirements

• Integration role

• Transformation history

• Compatibility

• Material uncertainty

A generated component may function independently but fail when integrated with other components.

AI-Generated Component Success ≠ Governed Composite Success.

SACM shall preserve the distinction between generated constituent outputs

and the resulting composition.

Autonomous Artifact Composition

Autonomous agents may select and assemble software artifacts.

SACM shall preserve the applicable composition boundaries and authority

references.

Relevant actions may include:

• Selecting components

• Generating missing components

• Modifying interfaces

• Resolving integration conflicts

• Combining artifacts

• Producing executable outputs

The agent's technical ability to assemble software does not establish authority to include every available component.

Autonomous Assembly Capability ≠ Autonomous Composition Authority.

Detailed autonomous execution control remains within ASCCM.

Multi-Agent Artifact Composition

Multiple agents may create separate components that are later integrated.

For example:

Agent A — Interface Component

Agent B — Business Logic Component

Agent C — Data Processing Component

Agent D — Integration Component

SACM shall preserve the material relationships between:

• Contributing artifacts

• Contributing actors

• Integration mechanisms

• Compatibility conditions

• Resulting composite artifacts

Successful coordination between agents does not automatically establish valid composition.

Multi-Agent Output Aggregation ≠ Governed Artifact Composition.

Human–AI Composition

Human developers and AI systems may contribute different components to the same software artifact.

SACM shall preserve material constituent relationships without assuming

that human-generated or AI-generated artifacts possess inherently superior governance status.

The relevant assessment concerns:

• Artifact identity

• Applicable authority

• Integrity evidence

• Compatibility

• Composition method

• Resulting artifact state

Human Inclusion ≠ Automatic Composition Validation.

AI Inclusion ≠ Automatic Composition Invalidity.

Generated Composite Artifacts

A software generation system may produce a complete application or package from multiple underlying inputs.

The final artifact may appear as a single generated output.

SACM shall identify material constituent relationships where they are relevant and sufficiently reconstructable.

These may include:

• Templates

• Source components

• Generated modules

• External libraries

• Intermediate artifacts

• Configuration inputs

• Assembly mechanisms

The existence of a single output file does not establish that the artifact has a single origin or constituent basis.

Single Output Artifact ≠ Single Constituent Origin.

Composition Transformation

Composition may transform constituent artifacts rather than merely place them together.

Examples include:

• Linking

• Compilation

• Code merging

• Interface adaptation

• Code generation

• Packaging

• Resource embedding

• Automated rewriting

SACM shall preserve material transformation relationships.

The governance chain is:

Constituent Artifact States → Composition Mechanism → Resulting Composite

Artifact State

Transformation evidence shall remain connected to relevant constituent

identities.

Successful Composition Transformation ≠ Governed Composition Integrity.

Composition Reproducibility

Reproducibility may support composition governance.

A reproducible composition process can provide evidence that defined constituent artifacts and conditions produce a corresponding composite output.

However, reproducibility does not independently establish:

• Legitimate inclusion authority

• Constituent origin validity

• Requirements satisfaction

• Software security

• Software quality

• Deployment readiness

Reproducible Composition ≠ Governed Composition Acceptance.

SACM shall preserve the actual evidentiary scope of reproducibility.

Composition Evidence

SACM shall preserve evidence sufficient to reconstruct material composition

relationships.

Relevant evidence may include:

• Composition objectives

• Constituent artifact identifiers

• Constituent versions

• Composition manifests

• Build or assembly records

• Integration decisions

• Transformation records

• Compatibility assessments

• Authority references

• Resulting artifact identifiers

• Exception records

The necessary evidence depends on the materiality and complexity of the

composition.

Composition Record Existence ≠ Composition Evidence Sufficiency.

Composition Manifest Governance

A composition manifest may identify the constituent artifacts and their relationships within a composite artifact.

SACM may use manifests to preserve:

• Constituent identities

• Versions

• Roles

• Inclusion relationships

• Source references

• Material transformation conditions

• Relevant integrity references

A manifest shall not automatically be treated as complete or accurate

merely because it was generated by a build tool.

Generated Manifest ≠ Verified Composition Reality.

Manifest evidence shall be assessed against the actual composition where material.

Composition Evidence Integrity

SACM shall evaluate whether the evidence supporting composition claims is sufficiently reliable.

Relevant considerations may include:

• Evidence source

• Evidence completeness

• Temporal relevance

• Content correspondence

• Transformation references

• Actor attribution

• Material contradictions

• Preservation conditions

Evidence generated after composition may support reconstruction but shall not automatically be treated as a contemporaneous record.

Reconstructed Composition Evidence ≠ Original Composition Evidence.

Where reconstruction is inferential, the uncertainty shall remain explicit.

Composition Uncertainty

Composition governance may remain uncertain where:

• Constituent artifacts cannot be fully identified

• Versions are missing

• Integration history is incomplete

• Compatibility assumptions are undocumented

• Generated outputs cannot be linked to inputs

• External components have uncertain origin

• Material transformation evidence is unavailable

• Composition manifests conflict with actual artifacts

SACM shall preserve material uncertainty.

Uncertain Composition ≠ Verified Composition.

Uncertainty in one constituent relationship shall not automatically

invalidate every other supported relationship.

Composition Conflict

Composition conflicts may arise when:

• Multiple component versions are incompatible

• Interface definitions contradict one another

• Constituent identity records disagree

• Build manifests differ from actual outputs

• Integration decisions conflict with requirements

• Multiple agents produce incompatible components

• Composition authority conditions contradict one another

SACM shall identify material conflicts and preserve the relevant evidence.

A technically successful build shall not automatically resolve governance

conflicts.

Build Success ≠ Composition Conflict Resolution.

Composition Gap

A Composition Gap exists where a material constituent, relationship, transformation, or evidence condition cannot be sufficiently established.

Examples include:

• Unknown constituent origin

• Missing component version

• Unrecorded artifact substitution

• Unexplained integration transformation

• Missing composition authority

• Incomplete composite artifact identity

• Unsupported compatibility assumptions

SACM shall identify the gap and its relevance to the resulting composite

artifact.

Composition Gap ≠ Universal Composite Invalidity by default.

Critical Composition Gap

A Critical Composition Gap exists where missing or contradictory composition evidence prevents a materially consequential composite software state from being reliably established.

Examples may include:

• A critical executable with unidentified embedded components

• A financial software package whose actual constituent versions cannot

• An autonomous system component assembled from unverified generated

• A composite artifact whose integration history contains unexplained

be reconstructed. artifacts. substitutions.

SACM shall preserve the critical gap and support competent escalation.

It does not independently determine operational suspension, release rejection, or liability.

Critical Composition Gap Identification ≠ Operational Decision Authority.

Composition Drift

Composition Drift occurs when the actual constituent structure materially differs from the recognized composition state.

Possible causes include:

• Unrecorded dependency changes

• Component replacement

• Regenerated artifacts

• New embedded resources

• Modified interfaces

• Build-process changes

• Automated integration

• Unauthorized component inclusion

SACM shall identify material drift and support reassessment.

Previously Governed Composition ≠ Permanently Governed Composition.

Composition Change and Recomposition

A composite artifact may be modified through:

• Addition of constituents

• Removal of constituents

• Replacement of components

• Interface changes

• Regeneration

• Recompilation

• Repackaging

• Structural reorganization

SACM shall preserve the relationship between previous and resulting

composition states.

A change to one constituent may affect multiple composition relationships.

Constituent Change ≠ Isolated Composition Change by default.

Detailed lifecycle change authorization remains within Software Change Management Standard (SCMS).

Composition Decomposition

A composite artifact may be decomposed into smaller software artifacts.

SACM shall preserve material decomposition relationships where relevant.

These may include:

• Original composite identity

• Extracted constituent identities

• Decomposition method

• Relevant transformations

• Resulting artifact states

• Supporting evidence

Decomposition shall not automatically establish that extracted components retain the same functional or governance meaning outside the original composition.

Extracted Component ≠ Independently Validated Component.

Composition Governance States

SACM shall support identifiable composition governance states.

Possible states include:

Proposed: A composition has been proposed.

Under Assessment: Material composition conditions are being evaluated.

Constituents Identified: Relevant constituent artifacts have been

identified.

Conditionally Supported: Composition recognition depends on explicit

conditions.

Composition-Supported: Material composition relationships are sufficiently supported.

Partially Supported: Some composition relationships remain insufficiently

established.

Evidence-Limited: Available evidence is insufficient.

Compatibility-Unresolved: Material compatibility conditions remain

uncertain.

Authority-Unresolved: Relevant composition authority cannot be

sufficiently established.

Conflicted: Material composition evidence or conditions contradict one

another.

Composition-Failed: A material composition condition has been determined

not to hold.

Superseded: A later governed composition state has replaced the earlier

state.

Unresolved: Available information does not support a sufficiently determined state.

The state shall distinguish composition evidence from functional validation

and release authorization.

Composition-Supported ≠ Software Product Accepted.

Composition Recognition

SACM shall determine whether the relevant composition relationships are sufficiently supported for their intended governance use.

Recognition should consider:

• Composite artifact identity

• Constituent artifact identity

• Constituent integrity references

• Composition authority

• Composition method

• Integration boundaries

• Compatibility

• Transformation evidence

• Resulting artifact identity

• Material uncertainty

• Composition conflicts

• Evidence sufficiency

Recognition shall remain limited to the composition dimensions actually

assessed.

Recognized Composition ≠ Universal Software Acceptance.

Composition Reassessment

SACM shall support reassessment when material composition conditions change.

Triggers may include:

• New constituent artifacts

• Component version changes

• Changed integration methods

• New compatibility evidence

• Artifact integrity changes

• Autonomous composition changes

• Build process modifications

• New requirements

• Composition drift

• Material evidence gaps

• Changes in composition authority

Reassessment shall preserve relevant historical composition states.

Composition Continuity

SACM shall preserve material composition relationships across changes in:

• Constituent versions

• Development actors

• Repositories

• Build environments

• AI models

• Integration tools

• Software packaging

• Organizational control

Continuity may be supported by composition manifests, artifact identifiers, transformation records, and evidence references.

It shall not be inferred solely from an unchanged application name or

version label.

Application Name Continuity ≠ Composition Continuity.

Composition and Downstream Lifecycle Governance

SACM shall provide relevant composition evidence to competent downstream

SLGA™ governance processes.

Such evidence may support:

• Dependency assessment

• Configuration assessment

• Software quality evaluation

• Software security evaluation

• Release governance

• Deployment governance

• Change impact analysis

• Retirement planning

SACM does not independently perform these downstream assessments.

Composition Evidence Availability ≠ Downstream Governance Completion.

Governed Software Artifact Composition State

The principal output of SACM is: Governed Software Artifact Composition State

The state shall preserve sufficient information to identify:

Composition Governance Object: The composition activity or artifact under

assessment.

Composition Identity: The relevant composition reference.

Composition Objective: The intended composite software outcome.

Requirements Reference: The applicable governed requirements basis.

Composite Artifact Identity: The resulting software artifact.

Composite Artifact Version: The relevant composition version.

Constituent Artifacts: The material participating artifacts.

Constituent Versions: The relevant artifact versions.

Constituent Roles: The role of each material constituent.

Constituent Integrity References: Applicable CRIM integrity states.

Composition Authority: Relevant authorization conditions.

Constituent Inclusion Authority: Material inclusion permissions.

Composition Method: The method used to combine artifacts.

Composition Scope: The permitted integration boundaries.

Composition Structure: The material constituent relationships.

Interface Relationships: Relevant integration interfaces.

Compatibility Conditions: Material compatibility assessments.

Dependency References: Relevant dependency participation.

Transformation History: Material composition transformations.

Composition Evidence: Records supporting reconstruction.

Manifest References: Relevant constituent manifests.

Evidence Integrity: The condition of supporting evidence.

Composition Gaps: Missing material composition relationships.

Critical Composition Gaps: Material unresolved composition conditions

requiring competent attention.

Conflicts: Contradictory composition evidence.

Uncertainty: Material unresolved assumptions.

Composition Drift: Material divergence from the recognized composition

state.

Composition Governance State: The current recognized composition

condition.

Continuity: Historical composition and recomposition relationships.

Downstream Handoff: Relevant information for development traceability and

lifecycle governance.

The state shall not independently imply software correctness, security,

release readiness, deployment authorization, or legal permission to use every included component.

Minimum Implementation Framework

A conforming implementation of SACM must provide the following six functional capabilities.

• Composite and Constituent Artifact Identity Governance

Capability to identify composite software artifacts, constituent artifacts,

versions, roles, and relevant composition objects.

The implementation must distinguish constituent identity from composite identity and preserve material version relationships.

• Composition Authority, Scope, and Inclusion Governance

Capability to identify composition authority, constituent inclusion

conditions, permitted integration scope, and relevant execution boundaries.

The implementation must not infer unrestricted inclusion authority from general permission to assemble software.

• Composition Structure, Compatibility, and Transformation Governance

Capability to represent material constituent relationships, integration

interfaces, compatibility conditions, composition methods, and transformation events.

The implementation must distinguish technical assembly success from governed composition validity.

• Constituent Integrity, Evidence, and Composition Recognition Governance

Capability to consume relevant artifact integrity states, preserve

composition evidence, assess resulting composition relationships, and identify material evidence limitations.

The implementation must not infer composite integrity solely from the integrity of individual constituents.

• Composition Change, Drift, Conflict, and Continuity Governance

Capability to identify composition changes, component substitutions,

compatibility conflicts, material gaps, drift, and recomposition relationships.

The implementation must preserve historical composition states and support reassessment.

• Governed Composition State and Lifecycle Handoff Governance

Capability to produce a Governed Software Artifact Composition State with

sufficient constituent, authority, structure, compatibility, evidence, uncertainty, and continuity information for competent software creation and downstream lifecycle governance.

The implementation must preserve material unresolved conditions during handoff.

GIE™ Compatibility

SACM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Composite artifact discovery

• Constituent artifact identification

• Artifact role classification

• Composition structure mapping

• Constituent inclusion analysis

• Composition authority reference mapping

• Integration boundary assessment

• Interface compatibility analysis

• Version compatibility analysis

• Constituent integrity state association

• Composition manifest analysis

• Generated artifact composition analysis

• Multi-agent artifact composition reconstruction

• Composition transformation mapping

• Composition evidence assessment

• Composition gap detection

• Critical composition gap identification

• Composition conflict analysis

• Composition drift detection

• Recomposition impact analysis

• Composition continuity assessment

• Governed composition state representation

• Development traceability handoff

GIE™ may infer candidate composition relationships or identify possible constituent conflicts.

Such analytical outputs shall remain distinguishable from verified

composition evidence and competent governance recognition.

GIE™ must not fabricate missing constituent relationships, treat a successful build as proof of composition legitimacy, or substitute technical assembly capability for the applicable composition governance requirements.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical SACM governance logic is:

Receive Governed Software Requirements Baseline

↓

Receive Governed Development Method Selection State

↓

Receive Relevant Code & Artifact Integrity States

↓

Identify Software Artifact Composition Governance Object

↓

Identify Intended Composite Software Artifact

↓

Establish Composition Objective

↓

Identify Material Constituent Artifacts

↓

Identify Constituent Versions and Roles

↓

Identify Composition Authority

↓

Assess Constituent Inclusion Authority

↓

Establish Composition Scope and Boundaries

↓

Identify Composition Method

↓

Establish Composition Structure

↓

Identify Material Constituent Relationships

↓

Assess Constituent Integrity References

↓

Assess Interface and Version Compatibility

↓

Identify Dependency Participation

↓

Assess Composition Transformations

↓

Identify Resulting Composite Artifact State

↓

Preserve Composition Manifest and Evidence

↓

Assess Composition Evidence Integrity

↓

Identify Composition Gaps and Critical Gaps

↓

Identify Compatibility Conflicts and Uncertainty

↓

Assess Composition Drift

↓

Determine Composition Governance State

↓

Establish Governed Software Artifact Composition State

↓

Preserve Composition Continuity

↓

Handoff to Development Traceability and Competent Lifecycle Governance

↓

Reassess When Material Composition Conditions Change

The principal output is:

Governed Software Artifact Composition State

Governance Outputs

SACM may produce:

Software Artifact Composition Governance Object

Composite Artifact Identity Record

Constituent Artifact Identity Record

Constituent Artifact Role Record

Composition Objective Record

Requirements-to-Composition Reference

Composition Scope Record

Composition Authority Reference

Constituent Inclusion Authority Record

Composition Method Record

Composition Structure Map

Constituent Relationship Record

Constituent Artifact Integrity Reference

Composite Artifact Integrity Reference

Composition Compatibility Assessment

Interface Compatibility Assessment

Version Compatibility Assessment

Composition Dependency Relationship Record

External Software Inclusion Record

AI-Generated Constituent Record

Autonomous Composition Record

Multi-Agent Composition Record

Human–AI Composition Record

Generated Composite Artifact Record

Composition Transformation Record

Composition Reproducibility Evidence

Composition Evidence Record

Composition Manifest

Composition Evidence Integrity Assessment

Composition Uncertainty State

Composition Conflict Record

Composition Gap Record

Critical Composition Gap

Composition Drift Record

Composition Change Record

Recomposition Record

Composition Decomposition Record

Composition Governance State

Composition Recognition Record

Composition Reassessment Record

Composition Continuity Record

Downstream Lifecycle Handoff Record

The principal output is:

Governed Software Artifact Composition State

Critical Distinctions

SACM preserves the following distinctions:

Technical Artifact Assembly ≠ Governed Software Artifact Composition

Constituent Artifact Identity ≠ Composite Artifact Identity

Product Name ≠ Composite Artifact Identity

Component Name ≠ Constituent Artifact Identity

Artifact Participation ≠ Fixed Artifact Role

Component Availability ≠ Composition Necessity

Working Composition ≠ Requirements-Aligned Composition

Composition Scope ≠ Unlimited Integration Authority

Artifact Access ≠ Composition Authority

Composition Authority ≠ Unlimited Constituent Inclusion Authority

Composition Method Availability ≠ Governed Composition Method Use

Artifact Collection ≠ Governed Composition Structure

Recorded Relationship ≠ Verified Composition Relationship

Constituent Inclusion ≠ Constituent Integrity Acceptance

Component Integrity ≠ Composite Artifact Integrity

Individual Component Functionality ≠ Composition Compatibility

Syntactic Interface Compatibility ≠ Semantic Interface Compatibility

Version Identification ≠ Version Compatibility

Dependency Inclusion ≠ Dependency Governance Acceptance

Public Availability ≠ Authorized Software Inclusion

AI-Generated Component Success ≠ Governed Composite Success

Autonomous Assembly Capability ≠ Autonomous Composition Authority

Multi-Agent Output Aggregation ≠ Governed Artifact Composition

Human Inclusion ≠ Automatic Composition Validation

AI Inclusion ≠ Automatic Composition Invalidity

Single Output Artifact ≠ Single Constituent Origin

Successful Composition Transformation ≠ Governed Composition Integrity

Reproducible Composition ≠ Governed Composition Acceptance

Composition Record Existence ≠ Composition Evidence Sufficiency

Generated Manifest ≠ Verified Composition Reality

Reconstructed Composition Evidence ≠ Original Composition Evidence

Uncertain Composition ≠ Verified Composition

Build Success ≠ Composition Conflict Resolution

Composition Gap ≠ Universal Composite Invalidity by default

Critical Composition Gap Identification ≠ Operational Decision Authority

Previously Governed Composition ≠ Permanently Governed Composition

Constituent Change ≠ Isolated Composition Change by default

Extracted Component ≠ Independently Validated Component

Composition-Supported ≠ Software Product Accepted

Recognized Composition ≠ Universal Software Acceptance

Application Name Continuity ≠ Composition Continuity

Composition Evidence Availability ≠ Downstream Governance Completion

Cross-Architecture Boundary

SACM governs software artifact composition within SCS and SLGA™.

It does not independently establish general authority, artifact origin integrity, autonomous execution conformance, dependency acceptance, software quality, security, or release authorization.

Within SCS:

DMSM — Development Method Selection Module governs the selection and

suitability of development methods.

CRIM — Code & Artifact Integrity Module governs artifact identity,

origin, transformation, and integrity.

ASCCM — Autonomous Software Creation Control Module governs autonomous

software creation execution and its authority boundaries.

SACM — Software Artifact Composition Module governs the formation,

integration, and recognition of composite software artifacts.

DTM — Development Traceability Module governs reconstructable creation

relationships across objectives, actors, activities, and artifacts.

Within SLGA™:

Software Requirements Standard (SRS) governs the requirements basis.

Software Dependency Standard (SDS) governs software dependencies.

Software Configuration Standard (SCFS) governs software configuration.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs release decisions.

Software Deployment Standard (SDPS) governs deployment conditions.

Software Change Management Standard (SCMS) governs broader lifecycle

changes.

Software Retirement Standard (SRTS) governs retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures may include:

GOA™ — Governance Architecture: General authority and delegation

boundaries.

AGA™ — Accountability Governance Architecture: Responsibility and

accountability relationships.

ASGA™ — Autonomous Systems Governance Architecture: Autonomous execution

governance.

AGI™ — Agent Governance Infrastructure: Agent identity, permission,

delegation, and coordination.

OBIDENITY® — Origin-Bound Identity Governance Architecture: Applicable

origin, identity, ownership, and provenance relationships.

ORA™ — Operational Reality Architecture: Competent operational reality

assessment.

SACM consumes relevant governed references without replacing the

responsibilities of these domains.

Canonical Boundary

SACM begins when multiple software artifacts are proposed or used to form an identifiable composite software artifact.

SACM governs:

• Software artifact composition objects

• Composite artifact identity

• Constituent artifact identity

• Constituent roles

• Composition objectives

• Requirements-to-composition alignment

• Composition scope

• Composition authority

• Constituent inclusion authority

• Composition methods

• Composition structure

• Constituent relationships

• Constituent integrity references

• Composite integrity references

• Compatibility conditions

• Interface compatibility

• Version compatibility

• Dependency participation

• External software inclusion

• AI-generated constituents

• Autonomous composition

• Multi-agent composition

• Human–AI composition

• Generated composite artifacts

• Composition transformations

• Reproducibility evidence

• Composition evidence

• Composition manifests

• Evidence integrity

• Composition uncertainty

• Composition conflicts

• Composition gaps

• Critical composition gaps

• Composition drift

• Composition change

• Recomposition

• Decomposition

• Composition governance states

• Composition recognition

• Composition reassessment

• Composition continuity

• Downstream lifecycle handoff

SACM does not independently determine:

• General legal or organizational authority

• Requirements legitimacy

• Development method suitability

• Complete constituent artifact integrity

• Autonomous execution authorization

• Complete development traceability

• Intellectual property ownership or licensing validity

• Software functional correctness

• Software security conformance

• Release readiness

• Deployment authorization

SACM ends when the relevant composition assessment has been represented as a:

Governed Software Artifact Composition State

with sufficient constituent identity, composition structure, authority

references, compatibility evidence, transformation history, uncertainty, and continuity for competent software creation and downstream lifecycle governance.

Use Case 1 — An Autonomous Agent Builds a Working Application Using an

Unapproved External Component

An organization authorizes an autonomous coding agent to create an internal

analytics application.

The agent generates several application components.

During development, it discovers an external software library that would accelerate implementation.

The library is publicly accessible and technically compatible with the application.

The agent incorporates the library into the software.

The resulting application compiles successfully.

Its functional tests pass.

However, the organization has not established whether the external

component is permitted for inclusion under the applicable development conditions.

SACM evaluates:

Creation Objective → Constituent Selection → Inclusion Authority →

Composition Method → Resulting Composite Artifact

The assessment identifies that the technical composition succeeded but the inclusion authority for one material constituent remains unresolved.

The composite artifact may receive the state:

Authority-Unresolved

until competent assessment establishes the applicable conditions.

Public Availability ≠ Authorized Software Inclusion.

Composition Authority ≠ Unlimited Constituent Inclusion Authority.

Build Success ≠ Governed Composition Acceptance.

SACM prevents technical integration success from substituting for

legitimate constituent inclusion governance.

Use Case 2 — Multiple AI Agents Produce Compatible Components but the

Final Artifact Cannot Be Reconstructed

A software organization uses a multi-agent development system.

Separate agents generate:

• User interface components

• Business logic

• Data processing functions

• Integration code

An integration agent assembles the outputs into a working application.

The application passes its initial tests.

However, the organization cannot sufficiently determine:

• Which versions of the generated components were included

• Whether all constituent artifacts correspond to the reviewed versions

• Which transformations occurred during integration

• Whether additional components were introduced

• Whether the final package matches the recorded composition manifest

SACM evaluates:

Constituent Artifacts → Versions → Composition Relationships → Integration

Transformations → Composite Artifact

The assessment identifies material gaps between the claimed composition

structure and the resulting software artifact.

The composition may receive the state:

Evidence-Limited

or:

Partially Supported

until sufficient evidence is established.

Multi-Agent Output Aggregation ≠ Governed Artifact Composition.

Generated Manifest ≠ Verified Composition Reality.

Component Integrity ≠ Composite Artifact Integrity.

SACM ensures that the ability to assemble functional software does not substitute for reconstructable constituent relationships.

Canonical Closing Statement

Modern software is rarely created as a single isolated artifact.

Applications, services, libraries, generated components, compiled outputs,

external packages, and autonomous development results are continuously combined into larger software systems.

The resulting software may function correctly even when its constituent relationships are incomplete, its integration history is uncertain, or its composition authority is unresolved.

A successful build does not establish that every included artifact was authorized.

A collection of integrity-supported components does not automatically establish the integrity of their combined result.

A generated composition manifest does not independently prove that the resulting software contains exactly the recorded constituents.

And efficient multi-agent assembly does not guarantee reconstructable software composition.

Technical Artifact Assembly ≠ Governed Software Artifact Composition.

Component Integrity ≠ Composite Artifact Integrity.

Composition Authority ≠ Unlimited Constituent Inclusion Authority.

Generated Manifest ≠ Verified Composition Reality.

Build Success ≠ Governed Composition Acceptance.

SACM establishes the methodological conditions under which software

artifacts become governed constituents of a composite software creation output.

It preserves the relationships between the parts, the mechanisms through which they are combined, and the resulting whole.

It does not require a particular development technology, composition framework, or actor.

It requires that material software composition remain identifiable, bounded, compatible, integrity-aware, and supported by reconstructable evidence.

SACM transforms software assembly from a technical integration activity

into a governed composition process — ensuring that the creation of a functional software whole never substitutes for legitimate constituent inclusion, supported artifact relationships, composition integrity, and reconstructable evidence of how the whole was formed.